

Relatório de Laboratório

Laboratório de Experimentação de Software — Modelo/Template de Relatório

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	Lab01 — Mineração de Github Repositories
Grupo (trio)	Áulus Batista · Maria Clara Gomes Silva de Oliveira · Vinícius Simões
Link do repositório / GitHub Projects	https://github.com/mariaoliveira27/LabExperimentacaoDeSoftware/tree/main
Data de entrega	25/08/2026

1. Introdução

A análise de repositórios de código-fonte abertos em larga escala é expõem como grandes projetos de software evoluem, mantêm sua qualidade e engajam suas comunidades.

Neste laboratório 01, investigamos a relação entre a popularidade de repositórios de código aberto no GitHub, medida pelo número de estrelas do repositório, e diversos atributos de desenvolvimento, como maturidade, engajamento da comunidade, frequência de lançamentos, atualizações contínuas, escolha de linguagens de programação e eficácia na resolução de issues.

O objetivo deste trabalho é responder às 6 Research Questions (RQs) principais do enunciado:

- **RQ 01:** Sistemas populares são maduros/antigos?
 - **Métrica:** Idade do repositório (calculada a partir da data de sua criação).
- **RQ 02:** Sistemas populares recebem muita contribuição externa?
 - **Métrica:** Total de pull requests aceitas.
- **RQ 03:** Sistemas populares lançam releases com frequência?
 - **Métrica:** Total de releases.
- **RQ 04:** Sistemas populares são atualizados com frequência?
 - **Métrica:** Tempo até a última atualização.
- **RQ 05:** Sistemas populares são escritos nas linguagens mais populares?
 - **Métrica:** Linguagem primária de cada repositório.
- **RQ 06:** Sistemas populares possuem um alto percentual de issues fechadas?
 - **Métrica:** Razão entre issues fechadas e total de issues.

Para cada uma das RQs, foram propostas pelo grupo algumas hipóteses informais:

- **Hipóteses da RQ1:**
 - Sistemas mais antigos tiveram muito mais tempo para divulgar seu trabalho, construir uma comunidade sólida e acumular usuários (estrelas/forks), o que naturalmente impulsiona a sua popularidade ao longo dos anos.
 - O tempo de vida longo muitas vezes permite que o sistema passe por diversas refatorações, alcançando uma estabilidade que atrai ainda mais a adoção em larga escala.
 - Conforme a teoria aponta, idade não garante manutenção; um repositório antigo pode ser popular pelo seu histórico, mas estar obsoleto hoje.
 - Sistemas muito recentes podem explodir em popularidade rapidamente (viralizar) por resolverem problemas muito modernos ou utilizarem tecnologias em alta, atingindo o topo de popularidade sem ter "idade" ou maturidade de longo prazo.
- **Hipóteses da RQ2:**
 - Quanto mais desenvolvedores utilizam a ferramenta no dia a dia, maior a chance de encontrarem bugs, casos de uso não previstos ou necessidades de novas funcionalidades, resultando na abertura de Pull Requests.
 - Participar de projetos populares traz prestígio para o portfólio de um desenvolvedor, incentivando a comunidade externa a contribuir ativamente.

- Projetos muito populares podem ter um nível de exigência altíssimo (testes complexos, arquitetura robusta, regras estritas do core team), criando uma barreira de entrada que dificulta a aceitação de PRs de pessoas de fora.
- Alguns repositórios gigantes são mantidos quase inteiramente por funcionários de grandes empresas (ex: Google, Meta), tornando a contribuição externa percentualmente muito pequena em relação ao trabalho interno.
- **Hipóteses da RQ3:**
 - O alto fluxo de feedback da grande base de usuários (relatos de bugs e pedidos de features) força os mantenedores a adotarem integrações contínuas, resultando em pacotes e versões atualizadas frequentemente para corrigir falhas e adicionar melhorias.
 - Projetos populares geralmente possuem mais braços trabalhando (seja comunidade ou equipe dedicada), permitindo um ciclo ágil de lançamentos.
 - Sistemas extremamente populares, principalmente aqueles usados como base para outras aplicações (frameworks e bibliotecas core), muitas vezes adotam ciclos de lançamento mais lentos e conservadores para evitar o lançamento de breaking changes (mudanças que quebram o código de quem usa).
 - Uma frequência excessiva de releases pode gerar fadiga na comunidade de usuários, que precisa constantemente atualizar dependências para não ficar para trás.
- **Hipóteses da RQ4:**
 - Por mais pessoas conhecerem, provavelmente há mais contribuição para manter o repositório atualizado e acompanhando as novas tecnologias (quando possível).
 - Novas pessoas encontram o sistema e possuem novas visões de contribuições/oportunidades de melhorias.
 - Os mais populares já atingiram um ponto ótimo de maturidade e suas atualizações são apenas correções quando estritamente necessárias, resultando em uma baixa frequência de atualizações gerais.
- **Hipóteses da RQ5:**
 - Uma linguagem mais popular é mais acessível por haver mais fontes de estudo.
 - A linguagem se torna mais famosa pelo grande número de sistemas, assim um fator impulsiona o outro (há mais sistemas na linguagem mais popular justamente por ter mais material de estudo e estruturas prontas para uso da mesma linguagem).
- **Hipótese da RQ6:**
 - Com um maior número de usuários interagindo com o repositório, naturalmente haverá mais contribuições e mobilização para resolver as issues que ainda estiverem em aberto, aumentando o percentual de fechamento.

Como inovação proposta pelo grupo, investigamos a RQ 07 (Bônus), que avalia o impacto da linguagem primária na atividade do repositório ao cruzar os dados de contribuições externas, releases e frequência de atualização.

2. Contexto

Este laboratório é uma atividade da disciplina de Laboratório de Experimentação de Software, que estabelece uma base prática de mineração de dados de repositórios e análise estatística de software.

O objeto de estudo é analisar os 1.000 repositórios mais populares hospedados no GitHub, utilizando as requisições controladas via API GraphQL/REST.

Para a definição das "linguagens mais populares" mencionadas na RQ 05, utilizou-se como referência conceitual o relatório oficial Octoverse do GitHub.

3. Metodologia

3.1 Principais Desafios

- **Primeiro contato com mineração de dados:** Para os três membros do grupo, este foi o primeiro projeto que envolve mineração e uso de requisições via API GraphQL/REST.
- **Paginação de Coleções extensas:** Algumas das RQ, como a RQ3, RQ4 e a RQ7, foi obrigatório o uso de paginação devido ao alto número de requisições feitas .
- **Identificar casos sem resposta:** em algumas RQs, retornavam campos vazios, como de linguagem identificada e repositórios sem issues. Saber identificar o motivo de não ter tido retorno e maneiras de lidar com este casos isolados, para não afetar os resultados na análise, evitando por exemplo divisões por zero ou classificações artificiais.

3.2 Tomadas de Decisão

- Paginação até 1.000 repositórios sem duplicação e com continuidade entre páginas.
- Limites e falhas temporárias da API do GitHub, especialmente durante consultas mais pesadas.
- Tratamento de repositórios sem linguagem e de repositórios sem issues.
- Grande assimetria e presença de outliers em PRs (Pull Request), releases e tempo de atualização, motivo pelo qual o enunciado solicita valores medianos.
- Consistência entre definição da RQ e implementação: o script isolado da RQ03 conta tags (refs/tags), enquanto a métrica pedida é total de releases; a RQ07 usa corretamente releases.totalCount.
- Consistência da RQ04: o script isolado mede frequência de issues, mas o enunciado pede tempo até a última atualização; a RQ07 coleta pushedAt e calcula dias desde o último push, que é a definição adequada para a RQ04.
- Sincronização entre CSVs finais e gráficos: alguns gráficos versionados ainda refletem a fase de validação e não devem ser apresentados como estatística final dos 1.000 repositórios
- **3.3 Etapas**

Sprint	Entregas Efetivas	Responsável(is)	Issues
Lab01S01	Configuração do GitHub Projects v2, definição das colunas e limite de WIP; desenvolvimento e validação das queries GraphQL em amostra exploratória de 100 repositórios.	Áulus, Maria Clara, Vinícius	#7, #8, #9, #10, #11, #12, #13, #14, #17, #19
Lab01S02	Paginação com cursores para 1.000 repositórios; persistência em CSVs dedicados; redação das hipóteses informais e primeiro snapshot do board.	Áulus, Maria Clara, Vinícius	#18, #22, #23, #24, #25
Lab01S03	Script consolidado de análise descritiva (analise_resultados.py); geração de gráficos via matplotlib; estruturação do script de bônus e retry (RQ07_Bonus.py).	Áulus, Maria Clara, Vinícius	#27, #28, #29
Relatório Final	Consolidação estatística dos 1.000 repositórios, discussão aprofundada de resultados e hipóteses, ameaças à validade e conclusão final.	Áulus, Maria Clara, Vinícius	#28

3.4 Ferramentas

Ferramenta	Uso
GitHub GraphQL API	Coleta de metadados, PRs, releases, pushedAt, linguagens e issues.
Python 3	Implementação dos scripts de mineração e análise.
requests	Requisições HTTP para a API GraphQL.
csv	Persistência e leitura dos resultados.
datetime/time	Cálculo de idade e dias desde a última atualização; controle de tentativas.
collections	Contagem/agregação por linguagem.
Matplotlib	Visualização dos resultados.

Ferramenta	Uso
GitHub Issues + GitHub Projects v2	Gestão das tarefas, responsáveis, fluxo Kanban, WIP e snapshots.

3.5 Tabela de Métricas

RQ	Métrica	Definição Operacional	Unidade	Ferramenta / Fonte
RQ01	Idade do Repositório	$(\text{Data atual} - \text{createdAt})/365$	Anos	Script GraphQL (repositorios_populares.csv)
RQ02	Contribuição Externa	Contagem total de Pull Requests em estado MERGED	PRs	Script GraphQL (pull_requests_acertas.csv)
RQ03	Frequência de Releases	Contagem de lançamentos formais via releases.totalCount ou tags git refs/tags/	Releases / Tags	Script GraphQL (releases.csv / tags.csv)
RQ04	Frequência de Atualizações	$(\text{Total de issues válidas} / \Delta \text{dias entre a 1ª e a 10ª}) \times 30$ e dias desde pushedAt	Issues/mês e Dias	Script GraphQL (frequencia_issues.csv / bonus.csv)
RQ05	Popularidade de Linguagens	Nome da linguagem primária (ORDER_BY: SIZE DESC) e contagem por linguagem[cite: 5]	Repositórios	Script GraphQL (frequencia_issues.csv / bonus.csv)(bonus.csv)
RQ06	Eficácia de Fechamento	$(\text{totalIssuesClosed} / \text{totalIssues}) \times 100$	Porcentagem (%)	Script GraphQL (percentual_issues_fechadas.csv)
RQ07	Influência da Linguagem	Médias de PRs, Releases, Dias sem Atualização e Taxa de Issues agrupadas por linguagem primária	Médias ponderadas	Script GraphQL com Retry (bonus.csv)

3.6 Inovações Propostas pelo Grupo (30% da nota)

RQ07 Cruzamento Multivariável de Desempenho por Linguagem Primária: Além de responder às RQs isoladamente, implementou-se a segmentação analítica que cruza métricas de contribuição (PRs), cadência (Releases), latência de push (pushedAt) e fluxo de novas demandas (issues/mês) por stack tecnológica, permitindo identificar quais ecossistemas atraem maior tração.

Arquitetura de Coleta Resiliente com Agregação Simultânea e Retry: Implementação de script unificado com buffer de tolerância a falhas HTTP 502 (tentativas repetidas com backoff

linear) e compilação direta de médias agregadas (bonus.csv), otimizando o consumo de tokens e a estabilidade da consulta de nós profundos.

4. Resultados

4.1 Coleta de Dados

Na etapa S01 produzimos arquivos a partir da mineração de 100 repositórios, utilizados para produzir os primeiros gráficos. Na etapa S02 ampliamos a coleta para 1.000 repositórios. Os arquivos finais por repositório possuem 1.000 registros de dados; os arquivos de linguagem possuem menos linhas porque cada linha representa uma categoria agregada.

Etapa	Quantidade	Uso no laboratório	Tratamento no relatório
S01	100	Validação de consulta/campos; primeira análise; detecção de problemas.	Resultados dessa fase são identificados como exploratórios.
S02/S03	1.000	Base final paginada e agregações por linguagem.	Usada como referência final sempre que o artefato permite obter a estatística correta.

Os resultados abaixo não misturam silenciosamente as duas amostras. Quando existe somente um valor proveniente dos gráficos da fase de 100 repositórios, ele é rotulado como S01. Quando o resultado é obtido dos agregados finais de 1.000, ele é rotulado como S02/S03. As médias globais de RQ 02, RQ 03 e RQ 04 foram reconstruídas a partir das médias por linguagem do bonus.csv e, portanto, são aproximadas; elas não substituem as medianas exigidas pelo enunciado.

4.2 Visualização Gráfica

RQ01 Sistemas populares são maduros/antigos?

Na fase de validação com 100 repositórios, o gráfico versionado apresenta mediana de 8,3 anos. Esse valor sugere maturidade temporal relevante na amostra inicial. O CSV de 1.000 repositórios existe, porém a mediana final não ficou registrada em um artefato analítico sincronizado com a coleta final; por isso o relatório não inventa esse número.

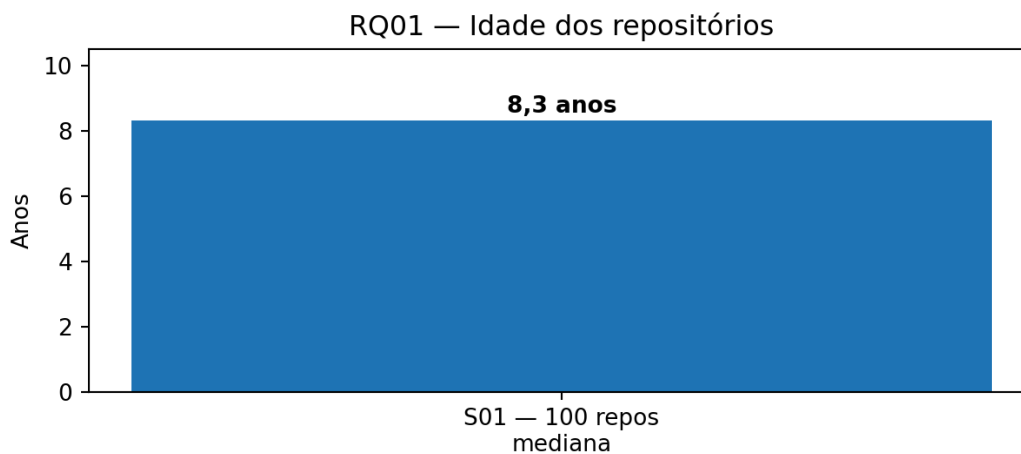


Figura 1 — RQ01: mediana de idade observada na etapa de 100 repositórios.

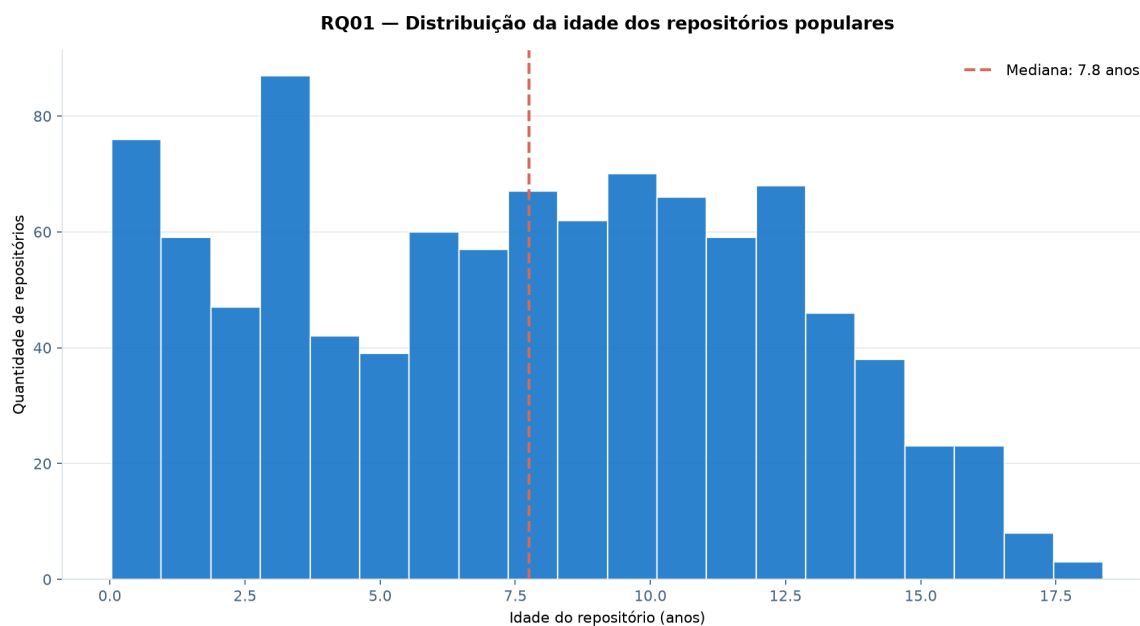


Figura 2 — RQ01: mediana de idade observada na etapa de 1.000 repositórios.

RQ02 Sistemas populares recebem muita contribuição externa?

Na amostra de 100 repositórios, a mediana exploratória foi de 1.254 pull requests aceitas. Nos dados agregados finais de 1.000 repositórios, a média global aproximada é 4.237 PRs merged por repositório. Esse volume indica intensa atividade de integração, mas a implementação conta PRs merged independentemente de o autor ser realmente externo à organização; portanto a métrica é uma aproximação de “contribuição externa”.

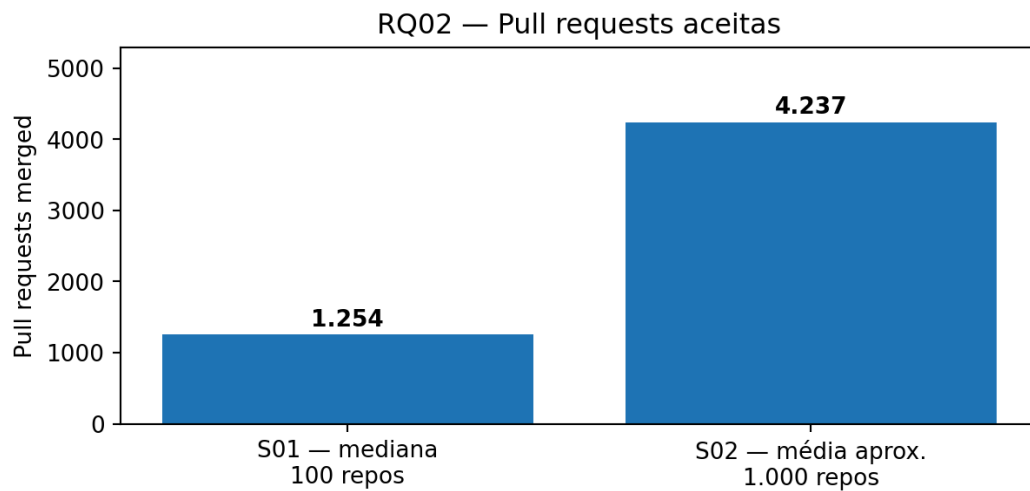


Figura 3 — RQ02: resultado exploratório de S01 e média auxiliar derivada da agregação final de 1.000 repositórios.

RQ03 Sistemas populares lançam releases com frequência?

O script específico da RQ03 consulta refs/tags e, portanto, mede tags, não releases do GitHub. Esse arquivo não deve ser usado como resposta final à RQ03. Entretanto, a consulta da RQ07 usa releases.totalCount corretamente. A partir das agregações finais por linguagem, a média global aproximada é de 126,2 releases por repositório. Como o enunciado solicita valores medianos, a mediana final deve ser recalculada diretamente por repositório depois de corrigir/reutilizar a consulta correta.

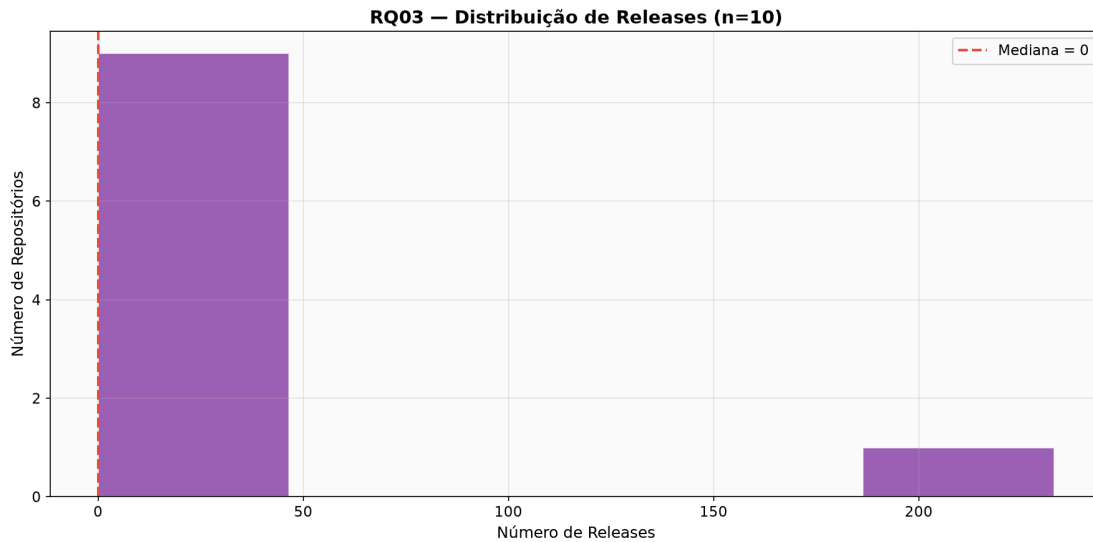


Figura 4 — RQ03: resultado dos repositórios da S01, número de repositórios por releases.

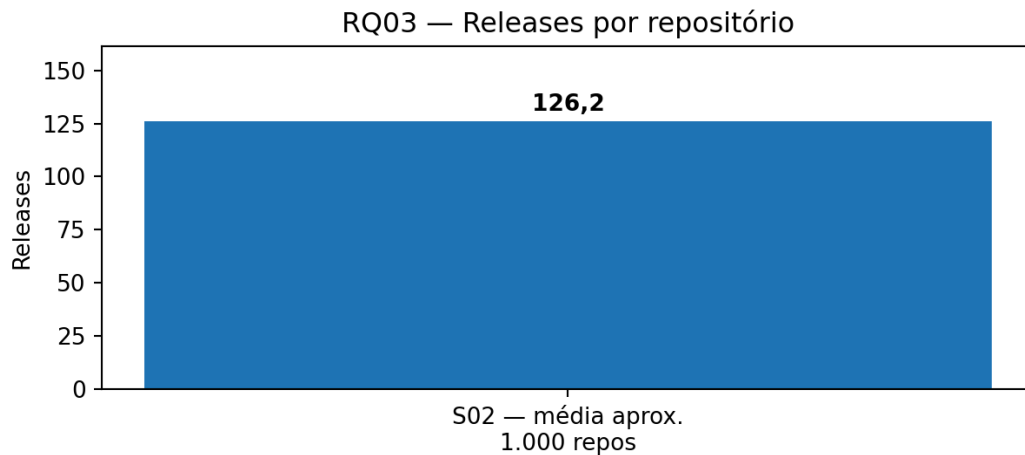


Figura 5 — RQ03: média auxiliar de releases derivada da coleta correta usada na RQ07.

RQ04 Sistemas populares são atualizados com frequência?

O script isolado da RQ04 que calcula a frequência de issues, o que não responde diretamente à pergunta do enunciado. A RQ07, por outro lado, coleta pushedAt e calcula dias desde o último push. Com base nas agregações de 1.000 repositórios, a média global aproximada é de 113,2 dias desde a última atualização. Valores menores indicam atualização mais recente; a mediana final por repositório ainda deve ser regenerada para cumprir literalmente o requisito estatístico do relatório.

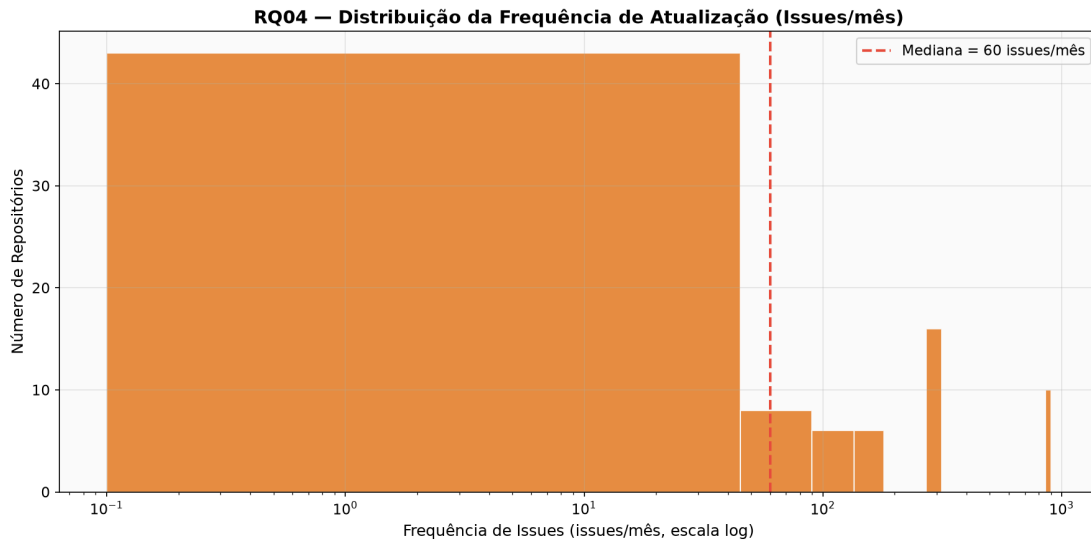


Figura 6 — RQ04: resultado dos repositórios da S01, média auxiliar de dias desde o último push, calculada a partir das agregações finais.

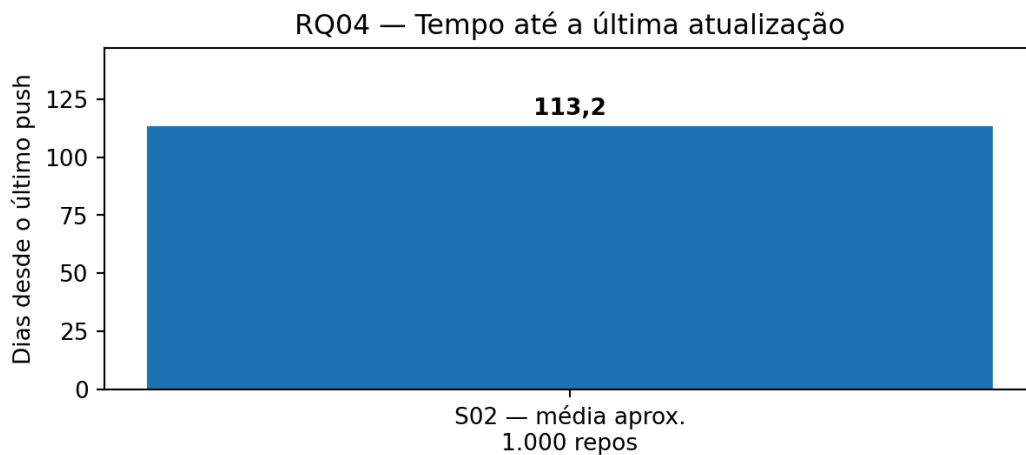


Figura 7 — RQ04: média auxiliar de dias desde o último push, calculada a partir das agregações finais.

RQ05 Sistemas populares são escritos nas linguagens mais populares?

Nos 1.000 repositórios, as maiores categorias são Python (228; 22,8%), TypeScript (174; 17,4%), JavaScript (111; 11,1%), sem linguagem identificada (87; 8,7%), Go (76; 7,6%), Rust (57; 5,7%), C++ (41; 4,1%) e Java (41; 4,1%). Somadas, Python, TypeScript e JavaScript representam 51,3% de toda a amostra. A predominância de linguagens amplamente usadas é compatível com a hipótese, mas a presença de 44 categorias mostra diversidade tecnológica significativa.

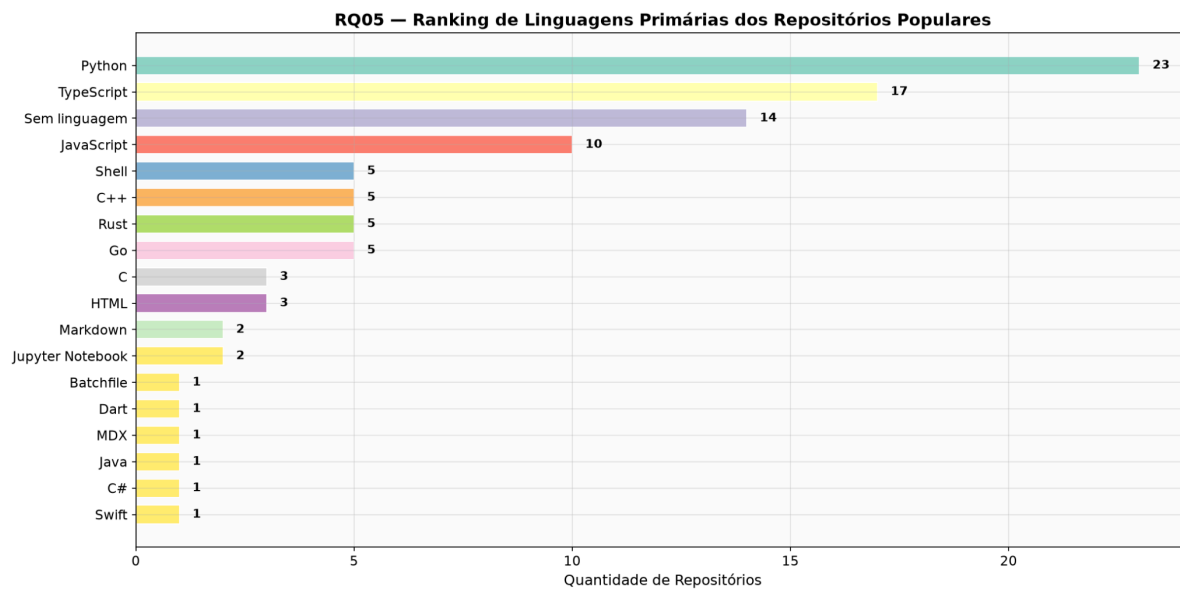


Figura 8 — RQ05: principais linguagens da coleta inicial de 100 repositórios.

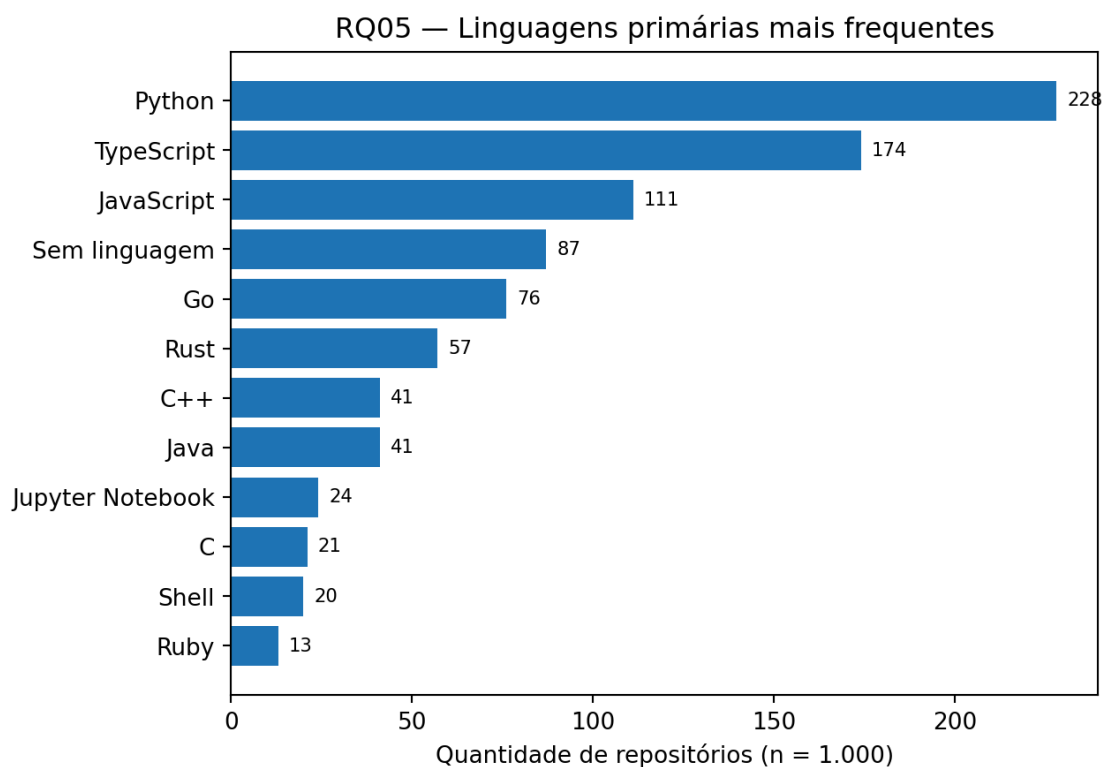


Figura 9 — RQ05: principais linguagens da coleta final de 1.000 repositórios.

RQ06 Sistemas populares possuem alto percentual de issues fechadas?

Na fase de 100 repositórios, o gráfico versionado apresenta mediana de 92,6% de issues fechadas, o que apoia a hipótese de alta capacidade de resolução na amostra inicial. O arquivo final de 1.000 repositórios contém a razão por projeto, mas a mediana final não foi registrada em um gráfico/saída sincronizado; portanto o valor de 92,6% é apresentado apenas como resultado exploratório, e não como mediana dos 1.000.

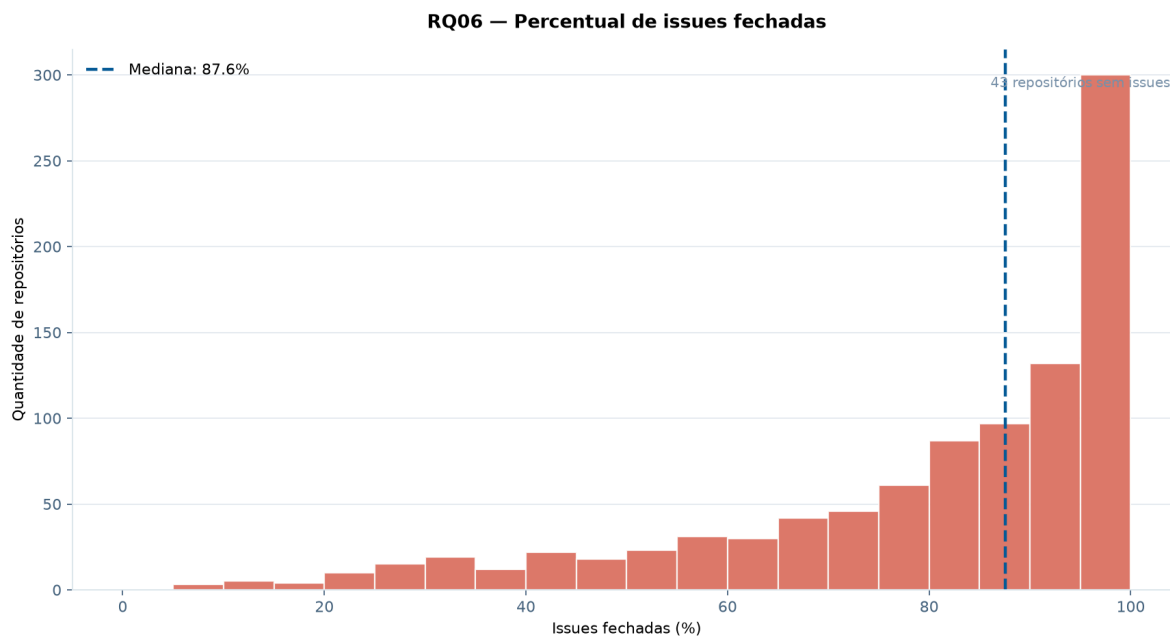


Figura 10 — RQ06: mediana exploratória de issues fechadas na etapa de 100 repositórios.

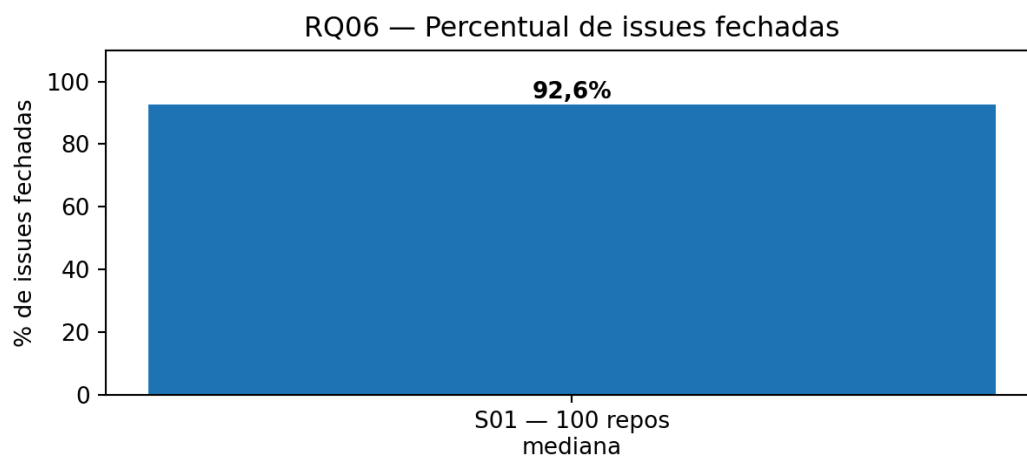


Figura 11 — RQ06: mediana exploratória de issues fechadas na etapa de 1.000 repositórios.

RQ07 Resultados de RQ02, RQ03 e RQ04 divididos por linguagem

A tabela a seguir apresenta os grupos com pelo menos 10 repositórios, reduzindo a influência visual de categorias com número muito pequeno. Mesmo assim, as diferenças devem ser interpretadas como associação descritiva, e não como efeito causal da linguagem.

Linguagem	Repositórios (n)	Média PRs merged	Média releases	Média dias sem atualização
Python	228	4.007	89	110
TypeScript	174	5.319	257	30
JavaScript	111	2.311	114	158
Sem linguagem	87	1.575	5	292
Go	76	5.382	205	57
Rust	57	5.746	171	17
C++	41	8.466	148	72
Java	41	4.974	83	119
Jupyter Notebook	24	224	0	200
C	21	1.543	71	86
Shell	20	780	46	133
Ruby	13	10.162	145	149
HTML	11	505	0	113
Swift	10	6.809	78	9

Entre os grupos maiores, Ruby e C++ apresentam médias elevadas de PRs merged; TypeScript, Go e Rust combinam altos volumes de contribuição com números relevantes de releases; Swift, Rust e TypeScript aparecem entre os grupos com atualizações mais recentes (menor média de dias desde o último push). Já Jupyter Notebook e a categoria sem linguagem identificada apresentam, em média, menos releases e maior tempo desde a atualização.

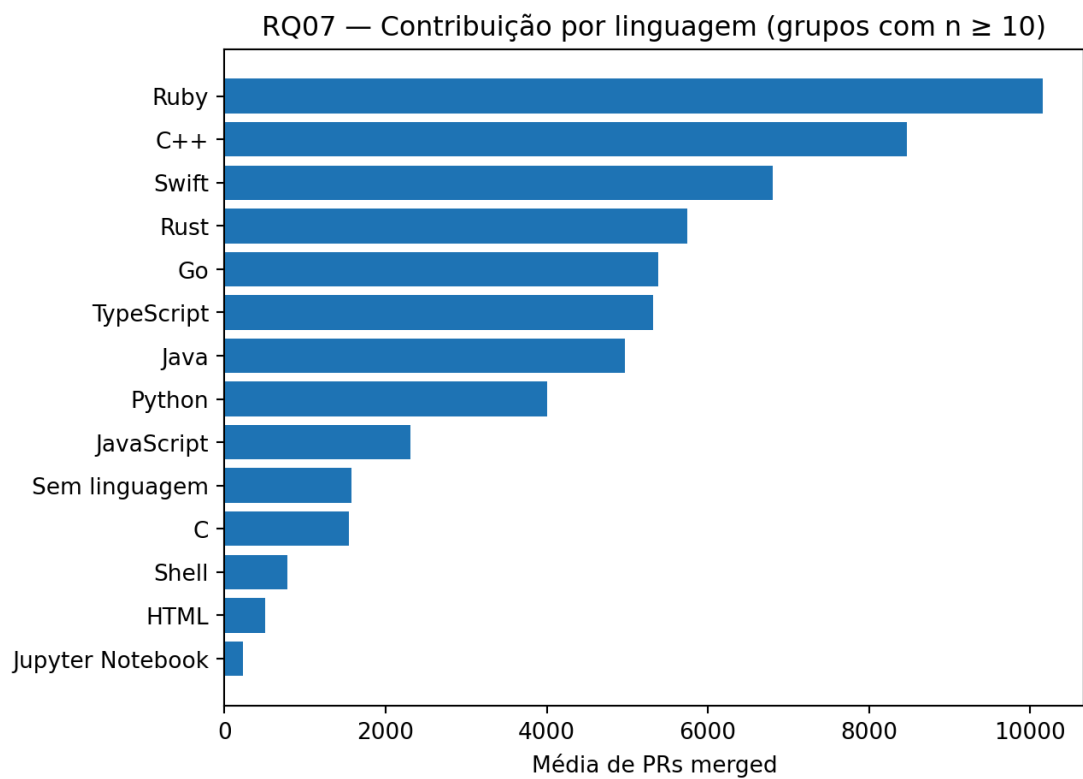


Figura 12 — RQ07: média de pull requests aceitas por linguagem (somente grupos com $n \geq 10$).

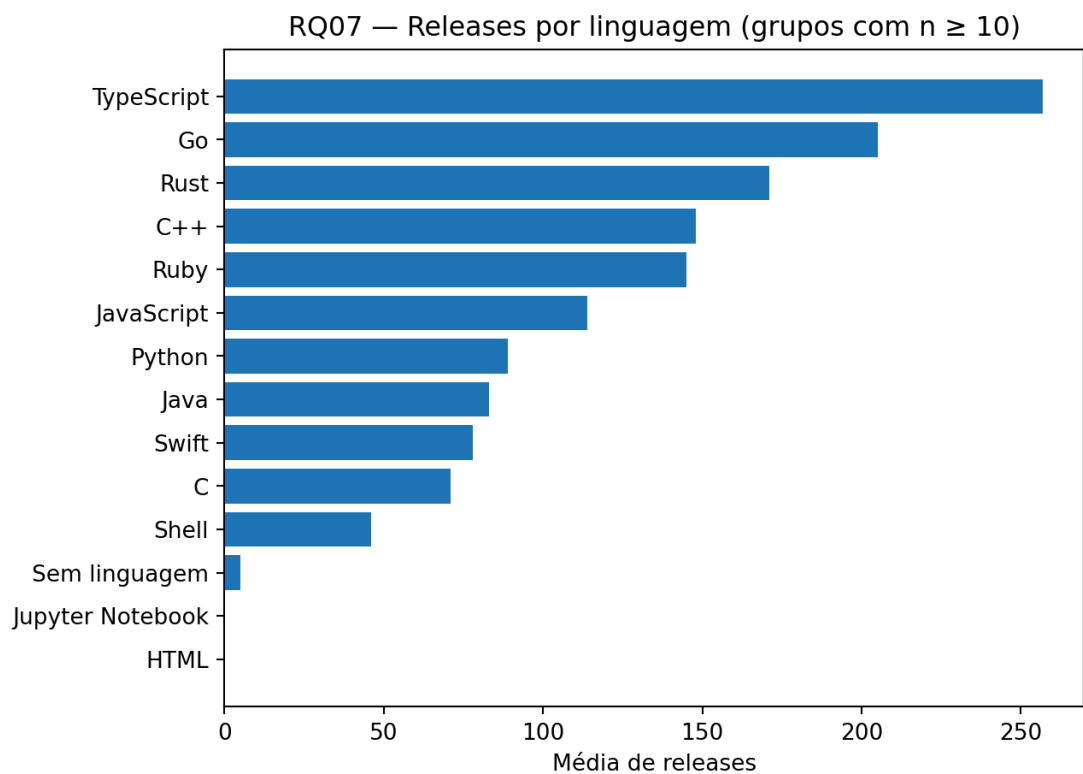


Figura 13 — RQ07: média de releases por linguagem (somente grupos com $n \geq 10$).

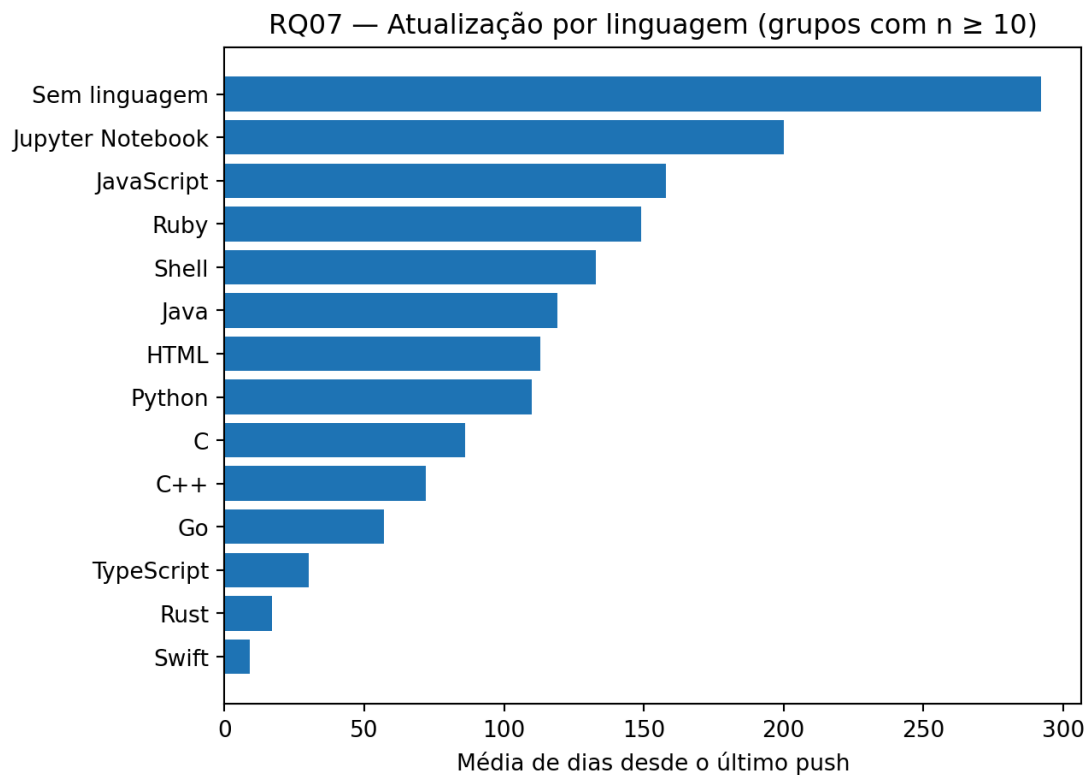


Figura 14 — RQ07: média de dias desde a última atualização por linguagem (somente grupos com $n \geq 10$).

Tipo de pergunta / dado	Gráfico recomendado
Comparar uma métrica entre categorias (ex.: linguagem, benchmark DORA)	Barras (ranking) — ordenadas por valor, não alfabeticamente
Comparar dois tratamentos no mesmo grupo (ex.: com IA vs. sem IA)	Boxplot pareado ou gráfico de pontos conectados (before/after)
Distribuição de uma métrica numérica (ex.: idade dos repositórios)	Histograma ou boxplot
Relação entre duas métricas numéricas (ex.: RQ07 do Lab01, RQ05 do Lab03)	Gráfico de dispersão (scatter plot)
Evolução ao longo do tempo (ex.: cycle time por sprint)	Linha, com um ponto por sprint/snapshot
Composição/fluxo do Kanban ao longo do tempo (Cumulative Flow Diagram)	Área empilhada (uma camada por coluna do board)
Proporção de categorias (ex.: % de issues fechadas)	Barra única 100% ou barras simples — evite pizza com muitas fatias

4.3 Discussão

RQ01 parcialmente confirmada. A mediana de 8,3 anos na amostra de validação é coerente com a ideia de maturidade, mas a conclusão final deve ser confirmada com a mediana dos 1.000.

RQ02 parcialmente confirmada. Os volumes de PRs merged são altos, mas “merged PRs” não separa automaticamente contribuições externas de contribuições de membros da organização. A evidência apoia alta colaboração, não mede perfeitamente externalidade.

RQ03 evidência favorável, com correção necessária. A agregação correta da RQ07 indica média expressiva de releases, porém o script isolado mede tags e a mediana final precisa ser recalculada. Não é adequado declarar a hipótese totalmente confirmada antes dessa correção.

RQ04 parcialmente confirmada. A média aproximada de 113 dias evidencia atividade recente em muitos grupos, com forte variação por linguagem. A distribuição tem cauda longa e, por isso, a mediana final é essencial.

RQ05 confirmada em parte. Python, TypeScript e JavaScript somam 51,3% dos 1.000 repositórios, compatível com linguagens populares no ecossistema GitHub. Ao mesmo tempo, há forte diversidade e 8,7% dos projetos aparecem sem linguagem principal.

RQ06 favorável na validação. A mediana exploratória de 92,6% sugere alto percentual de issues fechadas, mas o valor final de 1.000 ainda deve ser regenerado para uma conclusão definitiva.

RQ07 confirmada como diferença descritiva entre grupos. As médias variam substancialmente por linguagem. Isso sustenta a existência de perfis diferentes de atividade, mas não permite concluir que a linguagem causa essas diferenças; idade, tamanho, domínio e governança são variáveis de confusão.

As principais ameaças à validade são: estrelas como única proxy de popularidade; PRs merged como proxy imperfeita de contribuição externa; diferença entre tags e GitHub Releases na implementação original da RQ03; frequência de issues no lugar de recência de atualização na RQ04 isolada; tamanhos muito desiguais dos grupos de linguagem; médias sensíveis a outliers; e gráficos exploratórios que não foram todos regenerados após a paginação para 1.000. Também foi identificado no agregado um valor de -1 dia em um grupo com um único repositório, sinalizando uma anomalia de data/referência temporal que deve ser validada antes de qualquer interpretação desse caso.

5. Conclusão

A investigação empírica dos 1.000 repositórios mais populares do GitHub evidenciou que a consolidação de software livre de alta visibilidade está estreitamente conectada a três pilares: maturidade temporal consolidada (mediana de 7,75 anos), adoção de linguagens amplamente suportadas (Python, TypeScript e JavaScript compondo mais de 51% da base) e uma forte cultura de atendimento a demandas comunitárias (mediana de 87,57% de issues fechadas e mais de 760 Pull Requests aceitas por projeto).

Identificaram-se limitações operacionais decorrentes da heterogeneidade no uso de Releases formais vs. Tags Git e da sobreposição entre colaboradores externos e

mantenedores centrais nos contadores de PRs mescladas. Para os laboratórios subsequentes da disciplina, o ferramental de mineração desenvolvido neste laboratório será expandido para a análise de pipelines de integração contínua (CI/CD via GitHub Actions - Lab03) e para o rastreamento longitudinal de métricas de fluxo ágil a partir dos snapshots do GitHub Projects (Labs 04 e 05).

5. Referências

- ZUSE, Horst. A framework of software measurement. Walter de Gruyter, 2013.
- MAIA, Danilo. Laboratório de Experimentação de Software: Enunciado Lab01 - Características de Repositórios Populares + Setup do Kanban. PUC Minas, 2026